

PCA — Python Implementation Questions

1. Consider the following 2D data:

```
X = np.array([
    [2, 1],
    [3, 2],
    [4, 3],
    [5, 4],
    [6, 5]
])
```

Using Python:

- Calculate the mean of each feature.
- Subtract the mean from each feature.
- Display the centered data.

2. For the following data:

```
X = np.array([
    [1, 2],
    [2, 4],
    [3, 6],
    [4, 8],
    [5, 10]
])
```

Using Python:

- Calculate the mean of each column.
- Center the data.
- Plot the original data using a scatter plot.
- Observe the direction in which the data is distributed.

3. Using the centered data from Question 1:

- Calculate the covariance matrix using NumPy.
- Display the covariance matrix.
- Identify the variance of each feature from the covariance matrix.

4. Consider:

```
X = np.array([
    [1, 2],
    [2, 3],
    [3, 5],
    [4, 7],
    [5, 8]
])
```

Calculate:

- Mean of each feature
- Centered data
- Covariance matrix

5. Create two features in Python such that they are **strongly positively correlated**.

Calculate and display their covariance matrix.

What do you observe about the off-diagonal elements?

6. Create two features that are **negatively correlated**.

Calculate their covariance matrix and compare it with the covariance matrix of positively correlated features.

7. Given the covariance matrix:

```
C = np.array([
    [4, 2],
    [2, 3]
])
```

Using NumPy:

- Calculate the eigenvalues.
- Calculate the eigenvectors.
- Identify the eigenvector corresponding to the largest eigenvalue.

8. For the covariance matrix:

```
C = np.array([
    [5, 0],
    [0, 2]
])
```

Find the eigenvalues and eigenvectors.

Determine which direction represents the first principal component.

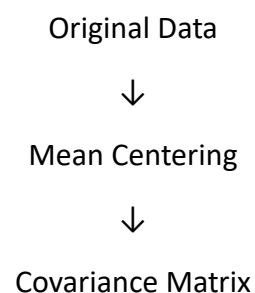
9. Given a covariance matrix:

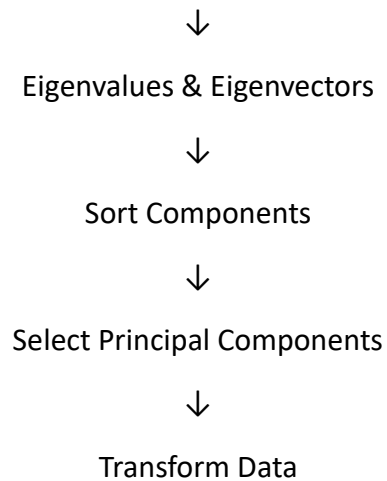
```
C = np.array([
    [6, 4],
    [4, 3]
])
```

Using Python:

- Find the eigenvalues.
- Find the eigenvectors.
- Sort the eigenvalues from largest to smallest.
- Rearrange the eigenvectors accordingly.

10. Implement the following PCA steps **without using sklearn**: [THIS YOU SHOULD DEFINITELY TRY OUT!]





Use the following data:

```
X = np.array([
    [2, 1],
    [3, 2],
    [4, 3],
    [5, 4],
    [6, 5]
])
```

11. Using the same dataset, project the centered data onto the **first principal component**.

Calculate the resulting one-dimensional representation.

12. Modify your PCA program so that the user can specify:

`n_components = 1`

and the program automatically:

- Selects the required eigenvectors.
- Projects the data.
- Returns the reduced data.

13. Use the following data:

```
X = np.array([
```

```
[2, 1],  
[3, 2],  
[4, 3],  
[5, 4],  
[6, 5]  
])
```

Apply:

```
from sklearn.decomposition import PCA
```

Reduce the data from **2 dimensions to 1 dimension**.

Display:

- Original data
- Transformed data
- Explained variance ratio

14. Apply PCA to the following 3-dimensional data:

```
X = np.array([  
[2, 1, 5],  
[3, 2, 6],  
[4, 3, 7],  
[5, 4, 8],  
[6, 5, 9]  
])
```

Reduce it from **3 dimensions to 2 dimensions**.

Display the transformed data.

15. For the dataset in Question 14:

- Calculate the explained variance ratio.
- Calculate the cumulative explained variance.
- Determine how much variance is retained using 1 component and 2 components.

16. Generate the following 2D data:

```
X = np.array([
    [1, 1],
    [2, 2],
    [3, 3],
    [4, 4],
    [5, 5],
    [6, 6]
])
```

Create:

1. A scatter plot of the original data.
2. A scatter plot after PCA transformation.

Compare the two plots.

17. Create a 3D dataset using:

```
X = np.array([
    [1, 2, 3],
    [2, 4, 6],
    [3, 6, 9],
    [4, 8, 12],
    [5, 10, 15],
    [6, 12, 18]
])
```

Apply PCA to reduce it from **3D to 2D**.

Create a scatter plot of the resulting two principal components.

18. Create a dataset where one feature has a much larger numerical scale than the other:

```
X = np.array([
```

[1, 100],

[2, 200],

[3, 300],

[4, 400],

[5, 500]

])

Apply PCA:

- Without standardization.
- After standardization using StandardScaler.

Compare the results.